Headless CMS Admin Panel

Walk us through it — architecture, data, and the honest parts

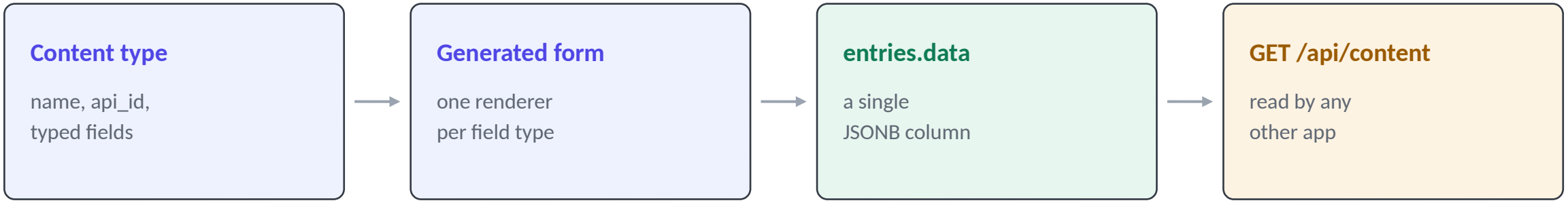
Jhon Ávila · Agile Monkeys Frontend Challenge 2026

237 TESTS | 11 DEFECTS FOUND | 5 MILESTONES

THE PRODUCT IN ONE SLIDE

Define a content type. Get a working editor.

There is no per-type form code anywhere in the repository. The editor is generated from the definition.



Change a field on a type that already has entries, and the save becomes a review:

- How many are affected
- What every value becomes
- Which ones will not convert
- What to do about those

Next.js 16 · React 19 · TypeScript · Tailwind v4 · Supabase Postgres + Realtime · Zod compiled at runtime

The backend is three tables, five indexes, one trigger and one stored procedure. The brief said keep it thin.

THE DECISION EVERYTHING FOLLOWS FROM

One JSONB column, not a table per type

NOT TAKEN

A real table per content type

Real columns, real types, real foreign keys. More honest about data — and it turns every schema change into DDL.

```
ALTER TABLE … RENAME COLUMN
ALTER TABLE … ALTER COLUMN TYPE
  USING <a cast nobody can preview>
```

You cannot preview a cast, row by row.

CHOSEN

Values in one JSONB column

A schema change stops being DDL and becomes an ordinary function from old rows to new rows.

```
transformEntry(entry, saved, draft)

  run it to render the preview
  run it to produce the write
```

That property is the whole feature.

What it costs, stated plainly

- No DB-level types or requiredness — validation is a Zod schema compiled at runtime, on every write
- A reference cannot be a foreign key — the check in the server action is a check, not a guarantee
- Ordering by a field sorts JSONB text, so ?order=price is lexicographic. Documented, not fixed

DATA MODEL

Three tables, and one asymmetry worth noticing

schemas

```
id · name
api_id  (unique, immutable)
description
```

1:N

fields

```
id · schema_id
key · label · type
required · position
target_schema_id
```

1:N

entries

```
id · schema_id
data  jsonb
invalid  boolean
```

The asymmetry

Deleting a type **cascades** to its own fields — but a type that is somebody else's reference target is `on delete restrict`. You cannot delete a type out from under one that points at it.

updated_at does double duty

It is also the optimistic-concurrency token. There is no separate version column — which is exactly why a migration must only rewrite rows whose data actually changed.

No authentication: the challenge scopes it out, so RLS is deliberately permissive. A demo posture, and the SQL says so.

ARCHITECTURE

Pure core, thin shell

Every decision is a pure function. Every side effect is a thin wrapper around one. That is why 237 tests cover the interesting behaviour without mounting a component or starting a database.

The pure modules

`diff.ts`	what changed, and how risky
`transform.ts`	what each stored value becomes
`analyze.ts`	counts, impacts, the write plan
`zod-builder.ts`	whether a value is valid
`sync-policy.ts`	connection state, refetch, conflict

Three ways data moves

Reads for the UI

Server Components query directly. No client fetch layer, no cache layer.

Writes

Server Actions. Validation is re-derived from the database, never from what the client sent.

Reads for other apps

Route Handlers. Public, CORS-open, no-store.

Real-time is not a fourth path: an event never carries data into the app.

THE GENERATED EDITOR

Validation compiled at runtime, from database rows

```
const VALIDATORS: Record<FieldType, (f: Field) => z.ZodTypeAny> = {
  text:      () => z.string().min(1, "This field cannot be empty"),
  number:    () => z.number({ message: "Enter a number" }).finite(…),
  boolean:   () => z.boolean(),
  date:      () => z.string().regex(ISO_DATE, …)
                   .refine(isRealCalendarDate, …),
  reference: () => z.string().uuid("Pick an entry"),
};
```

Why isRealCalendarDate exists

`Date.parse("2026-02-31")` does not fail. It rolls over to 3 March.

A regex-shaped date validator that trusts Date.parse accepts 31 February and stores it. The check round-trips through Date.UTC and compares.

Adding a sixth field type

No per-schema work — that is the property the brief asked for, and it holds. But the honest count of files is six, not the two an in-code comment claimed:

`FIELD_TYPES` `VALIDATORS` `renderer registry` `labels + matrix` `convertValue` `SQL enum`

Four of those are exhaustive Record<FieldType, …> maps — the build breaks until they are all filled in.

The compiler enumerates the work rather than letting a half-added type ship. I found the wrong comment writing this deck.

REAL-TIME

One rule

Events say what changed. The server says what it now is.

An event never patches client state from its payload. It triggers router.refresh() and the server re-renders. Chattier than diffing locally, and it removes the entire class of bug where a client's copy silently drifts from the database.

Four states, and the distinctions are deliberate

`connecting`	first attempt — a failure here is not a lost connection
`live`	subscribed
`reconnecting`	was live, lost it, retrying — not the same as offline
`offline`	channel closed, or the browser says the network is gone

Coming back from a gap

Any transition into live from reconnecting or offline triggers a full refetch, not a replay. During a gap the database is the only thing that knows the truth.

250 ms debounce

A schema migration touches every entry of a type. Without the debounce that is one refetch per row, in every open client.

reconnecting is separate from offline because telling someone they are disconnected mid-retry is alarmist, and wrong.

WHERE REAL-TIME MET CONCURRENCY

The bug that would have eaten a colleague's work

```
// components/entry/entry-form.tsx
const [baselineUpdatedAt, setBaselineUpdatedAt] =
  useState(entry?.updated_at ?? "");
```

`useState` — deliberately ***not*** derived from the prop.

1. **Real-time fires** a colleague saves the same entry
2. **router.refresh()** this page re-renders with a NEW updated_at
3. **If the token tracked the prop** their timestamp becomes my baseline
4. **My next save matches** and overwrites their work — reporting success

How it was caught

Not by a test — no test would have caught it. By asking what the feature I had just built does to the feature I built two milestones earlier.

That is a question about the system, not about the diff in front of you.

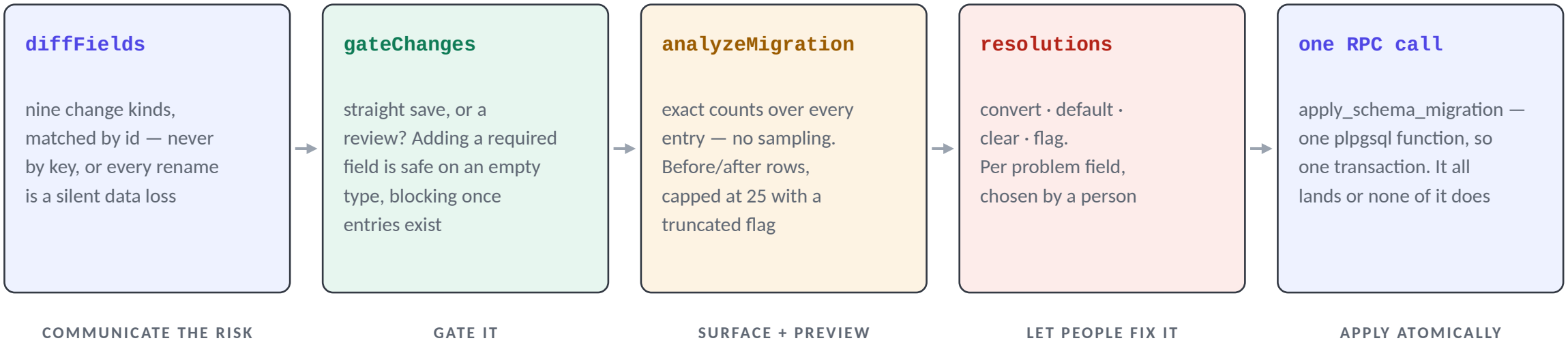
And when it does conflict

The user gets an explicit "Overwrite anyway" button wired to a separate server action — never a retry flag. An overwrite should always be a second deliberate decision.

Freezing it against the prop — it moves only when this form itself saves — is what keeps the conflict detectable.

SCHEMA EVOLUTION — THE CORE OF THE CHALLENGE

Communicate · surface · preview · fix · apply



Only what changed gets written. Entries are compared key-by-key, not with `JSON.stringify`, whose equality is key-order sensitive. Without that, reordering fields in the UI would have rewritten every row, bumped every `updated_at`, and woken every connected client — for a purely cosmetic change.
Fields are matched by id, so a rename remaps a JSONB key rather than dropping one and creating another. Match by key instead and every rename is a silent delete — which is why that one line is the load-bearing part of the whole feature.

Four of those five captions are the brief's own words. Each one is a stage, not a slogan.

WHY THE PREVIEW CAN BE TRUSTED

The migration engine computes nothing

The tempting design is to do the conversion in PL/pgSQL — it is right there next to the data. That means two implementations of the same rules, and any drift produces a preview that lies, which is worse than having no preview.

```
preview  --+
           +-->  transformEntry()  -->  the values
write    --+

the SQL only writes what was already computed
```

A test asserts that property directly — describe("preview and apply agree").

Three things in that function worth a reviewer's attention

The unique constraint must be deferrable — swapping two field keys is legal at the end of the batch and illegal halfway through it

It refuses rather than guesses — an incomplete field list raises — that is somebody editing the type in another tab

Constraints go back to immediate before it returns — so a violation surfaces here, not at COMMIT where nobody can attribute it

I WOULD FLAG THIS

Two matrices, one question

diff.ts holds the risk labels shown in the UI. transform.ts holds the actual conversion. They are separate implementations and can disagree at the margins.

text → reference is labelled lossy, but a well-formed UUID converts fine.

That direction is safe — warn more than you break — but nothing enforces the direction. First thing I would add: a property test asserting the advisory matrix is never more optimistic than the executable one.

TRADE-OFFS

What each decision bought, and what it cost

DECISION	BOUGHT	COST
JSONB entry storage	Schema evolution as a previewable data transform	No DB-level types or reference integrity
Transform in TypeScript, not PL/pgSQL	Preview and write cannot disagree	The migration is not a pure SQL artefact
Refetch on event, never patch	No client/server drift; open forms keep their values	Chattier; needs the 250 ms debounce
updated_at as the concurrency token	No extra column, no version bookkeeping	Any write bumps it — hence "only write what changed"
Server Actions instead of a write API	No client fetch code, no duplicated validation	The public API is read-only by construction
No component library	Nine primitives, native accessibility, no Radix tree	Everything hand-rolled, including focus states
Semantic colour tokens everywhere	A dark theme cost zero component edits	Enforced by convention — six literals had crept in

No authentication, permissive RLS: scoped out by the brief, which kept the timebox on schema evolution.

THE AI SESSION RECORD

Use AI. Own every line.

I treated the AI as a fast implementer with no judgement about this product, and kept three things: what to build, what "done" means, and whether the result is actually true.

I owned

Scope and priorities · architectural decisions and their trade-offs · what counts as verified · accepting or rejecting every diff

The AI owned

Turning a decided approach into code · boilerplate: types, forms, SQL scaffolding · writing tests for cases I named, then finding more

The rule: no code merged on the basis that it looked right.

Run the SQL
against a real Postgres 16

Call the API
curl, not only unit tests

Drive the UI
headless Chromium

Stub what you cannot reach
a Phoenix-protocol websocket server

Roughly a third of the effort went into proving things rather than writing them. That is the price of working this fast.

THE HONEST PART

Eleven defects — all in code that looked correct

3

Edge semantics

Impossible dates, ambiguous conversions

2

Failure paths

Leaked internals, wrong status code

2

Second runs

Non-idempotent migration, silent type collapse

4

Concurrency & the DOM

Overwrite-on-refresh, effect violations, a stale media listener, hover beating checked

The pattern: reliable at structure, weak at edge semantics. It produces a well-organised date validator that accepts 31 February.

Almost every one sits at a boundary — an impossible value, a failure path, a second run, two things happening at once.

Every one of these passed review by eye. None would have been caught by reading the diff.

WHAT I WOULD IMPROVE NEXT

Ordered by what I would actually do first

1 Close the two-matrix gap

A property test asserting the advisory risk matrix is never more optimistic than the executable one. An hour of work; removes the last place two sources of truth about one question survive.

2 Test the write path

lib/actions has no unit tests — the pure logic it calls is covered, the orchestration is not. Integration tests against a real Postgres, not mocks.

3 Clear three rough edges

The review offers "clear" where it cannot help. A partial index has no query behind it. Both surfaced writing this deck.

4 Make migrations scale past one page

Above 5,000 entries it refuses outright rather than migrating a prefix. Refusing is correct; stopping there is not.

5 Undo

A migration is atomic but irreversible. The before-values are already computed for the preview — and currently thrown away.

6 Real reference integrity

A references table alongside the JSONB, so the database can answer "what points at this?" without a scan.

Then, and only then, the product features — publishing states, drafts, i18n, media. All easier once a migration can be undone.

IN ONE PARAGRAPH

I chose a storage model that trades database-enforced typing for the ability to treat a schema change as an ordinary, previewable function over data — then spent the rest of the build making that function the single source of truth, so what the review screen promises and what the transaction writes cannot come apart.

Real-time is deliberately dumb: events say something changed, the server says what it is now. The one place that had to be clever about it — the concurrency token — is the place a plausible implementation silently loses a colleague's work.

237
tests, 8 files

2,423
lines of test

8,267
lines of source

12
routes, build clean

Nothing here requires taking my word for it — docs/WALKTHROUGH.md maps every claim to the file that proves it.